aspect to consider while debugging memory issues. If only one collection object is expected to contain (hold references to) instances of a particular object, but in the dominator view, if that collection object's retained size does not include the retained size of these instances, then it can be concluded that another object also is holding a reference to those objects. The other object that is holding these references could be preventing a garbage collection of these objects, thus resulting in a memory leak.

Figure 3: MAT object inspector

Figure 4: MAT opening dominator

The dominator tree can be opened by clicking on the *Dominator Tree* link in the bottom half of the *Overview* page.

The objects present on the heap, traced from their GC root, are listed in the dominator tree. In the dump being considered, the largest retained heap size originates from the main thread. This thread has a reference to the instance of *com.perf.memory.MemoryHogger*. This object, in turn, has a reference to an instance of *java.util.ArrayList* that is backed by an object array. This array is the object that consumes maximum heap space. Drilling down into this object array reveals that it holds a large number of objects of type *MyBigObject*. This object contains another object of type *TestObject* and so on. The shallow and the retained size of each object is shown in the dominator tree view.

While the dominator tree view gives a graph view of objects by their retained size from the root, the histogram view lists the objects based on the instance count, shallow size and retained size. By default, neither the dominator tree view nor the histogram view differentiate between classes loaded by the JVM or classes loaded by the bootstrap class loader. Due to this, it will be very difficult to find out the objects created by the applications that are consuming memory vs the objects created by JVM itself. MAT provides a feature to group the objects by class loader. This feature is very useful in analysing the objects created by the program. Though this feature is available in all the major views, it is less useful in the dominator tree view because the dominating objects of the application program could have been created by the system class loader. For instance, in the above example, the GC root of the object consuming maximum memory – the '*list*' object – is the main thread of type *java.lang.Thread.* Since this thread class is loaded by the system class loader, the dominator tree view

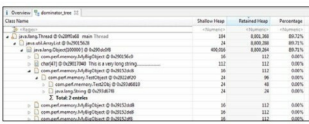

Figure 5: MAT dominator tree view

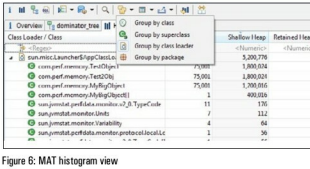

Figure 6: MAT histogram view

shows the '*list*' object in the system class loader group.

The grouping feature is much more helpful in the histogram view, where the objects loaded by the application can be viewed readily by looking at the application class loader's objects.

This feature of grouping by class loader is more useful when analysing memory dumps from an application server such as Websphere. In most typical configurations, the class loader isolation policy for Web and enterprise applications is set and, hence, each application gets its own class loader. In this situation, even though code in different applications could be in different packages, classes for supporting functions such as logging could be common across all applications. In this kind of situation, it is very important to find out to which application a particular object belongs to. Grouping by class loader comes in very handy in troubleshooting memory issues in these conditions.

An interesting observation from the histogram in Figure 6 is that the objects do not show their parent or child references. To get those references, right click an *object* and choose *Merge Shortest Paths to GC*. This action shows which GC root is holding a reference (direct or via another object) to this object. This is essentially the same as going back to the dominator tree view, but for the selected object alone.

Note: MAT reports the memory sizes after aligning the object sizes on the 8 byte boundary. This is in contrast to JVisualVM, which reports just the object size without aligning it along the 8 byte boundary. Due to this, JVisualVM may report lower-than-actual memory usage by objects.

A couple of utilities are available in MAT to show the memory wastage in the application. The Java collections utilities show the wastage in collection objects and the hash collisions